Министерство образования Республики Беларусь
Учреждение образования «Белорусский государственный университет информатики и радиоэлектроники»

Факультет компьютерных систем и сетей
Кафедра информатики
Дисциплина «Методы оптимизации и управления»

ОТЧЕТ
к лабораторной работе №6
на тему:
«ТРАНСПОРТНАЯ ЗАДАЧА»
БГУИР 6-05-0612-02 67

Выполнил студент группы 353503
КОХАН Артём Игоревич

(дата, подпись студента)

Проверил доцент каф. ИКТ
ГОРБУКОВ Дмитрий Михайлович

(дата, подпись преподавателя)

Минск 2026

СОДЕРЖАНИЕ

1 Индивидуальное задание 3
2 Выполнение работы 4
Вывод 6
Список использованных источников 7
Приложение А (справочное) Исходный код 8

1 ИНДИВИДУАЛЬНОЕ ЗАДАНИЕ

Реализовать метод решения транспортной задачи. Необходимо проверить совместность задачи по условию баланса и неотрицательности, построить начальный базисный допустимый план перевозок методом северо-западного угла и затем улучшать его до оптимального.

В соответствии с условием лабораторной работы в программе должны быть реализованы вычисление потенциалов, поиск небазисной позиции с отрицательной оценкой, построение цикла после ввода новой базисной позиции и пересчет плана перевозок по найденному циклу.

2 ВЫПОЛНЕНИЕ РАБОТЫ

В программе реализованы функции построения начального плана методом северо-западного угла, вычисления потенциалов u и v, выбора входящей позиции, построения единственного цикла и обновления плана перевозок. Также вычисляется суммарная стоимость перевозок.

Для примера из методических указаний используется матрица стоимостей c = [[8, 4, 1], [8, 4, 3], [9, 7, 5]], вектор поставок a = [100, 300, 300] и вектор заявок b = [300, 200, 200]. Условие баланса выполняется, поэтому задача совместна.

Метод северо-западного угла строит начальный базисный допустимый план x = [[100, 0, 0], [200, 100, 0], [0, 100, 200]] и базис B = {(1, 1), (2, 1), (2, 2), (3, 2), (3, 3)}. Для него вычисляются потенциалы u = [0, 0, 3] и v = [8, 4, 2]. Первая отрицательная небазисная оценка соответствует позиции (1, 3), для которой delta = -1.

После построения цикла [(1, 3), (1, 1), (2, 1), (2, 2), (3, 2), (3, 3)] и выбора theta = 100 получается новый план x = [[0, 0, 100], [300, 0, 0], [0, 200, 100]]. На следующих итерациях в базис последовательно вводятся позиции (3, 1) и (2, 3), после чего план перевозок улучшается до вида [[0, 0, 100], [0, 200, 100], [300, 0, 0]].

После четвертой проверки все небазисные оценки становятся неотрицательными, поэтому найденный план является оптимальным. Итоговый базис равен {(2, 2), (3, 3), (1, 3), (3, 1), (2, 3)}, а минимальная стоимость перевозок составляет 3900. Результат работы программы приведен ниже на рисунке 2.1.

```
--- Итерация улучшения 2 ---
u = [0.0, 1.0, 4.0]
v = [7.0, 3.0, 1.0]
В базис входит позиция (3, 1), delta = -2.0
Цикл = [(3, 1), (3, 2), (2, 2), (2, 1)]
theta = 200.0
Из базиса выходит позиция (3, 2)
Новый план перевозок =
[0.0, 0.0, 100.0]
[100.0, 200.0, 0.0]
[200.0, 0.0, 100.0]
Новый базис B = [(2, 1), (2, 2), (3, 3), (1, 3), (3, 1)]

--- Итерация улучшения 3 ---
u = [0.0, 3.0, 4.0]
v = [5.0, 1.0, 1.0]
В базис входит позиция (2, 3), delta = -1.0
Цикл = [(2, 3), (2, 1), (3, 1), (3, 3)]
theta = 100.0
Из базиса выходит позиция (2, 1)
Новый план перевозок =
[0.0, 0.0, 100.0]
[0.0, 200.0, 100.0]
[300.0, 0.0, 0.0]
Новый базис B = [(2, 2), (3, 3), (1, 3), (3, 1), (2, 3)]

--- Итерация улучшения 4 ---
u = [0.0, 2.0, 4.0]
v = [5.0, 2.0, 1.0]
Все небазисные оценки неотрицательны. План оптимален.

--- Результат ---
Оптимальный план перевозок =
[0.0, 0.0, 100.0]
[0.0, 200.0, 100.0]
[300.0, 0.0, 0.0]
Оптимальный базис B = [(2, 2), (3, 3), (1, 3), (3, 1), (2, 3)]
Минимальная стоимость = 3900.0
```

Рисунок 2.1 – Результат работы программы

ВЫВОД

В лабораторной работе реализован метод решения транспортной задачи с построением начального базисного допустимого плана и последующим улучшением по потенциалам и циклам. На примере из методических указаний программа корректно получила оптимальный план перевозок и минимальную стоимость 3900. Это подтверждает правильность реализации метода северо-западного угла и процедуры перехода к следующему базисному плану.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

[1] Python Documentation [Электронный ресурс]. – Режим доступа: https://docs.python.org/. – Дата доступа: 09.04.2026.

[2] NumPy Documentation [Электронный ресурс]. – Режим доступа: https://numpy.org/doc/. – Дата доступа: 09.04.2026.

ПРИЛОЖЕНИЕ А
(справочное)
Исходный код

Листинг А.1 – Реализация метода решения транспортной задачи

```
from collections import deque

TOL = 1e-9

def copy_matrix(matrix):
    return [row[:] for row in matrix]


def round_vector(vector, digits=6):
    return [round(x, digits) for x in vector]


def round_matrix(matrix, digits=6):
    return [round_vector(row, digits) for row in matrix]


def print_matrix(name, matrix):
    print(f"{name} =")
    for row in round_matrix(matrix):
        print(row)


def northwest_corner_method(supply, demand, verbose=True):
    """
    Метод северо-западного угла.
    """
    a = supply[:]
    b = demand[:]
    m = len(a)
    n = len(b)

    x = [[0.0 for _ in range(n)] for _ in range(m)]
    basis = []

    i = 0
    j = 0

    while True:
        basis.append((i, j))
        amount = min(a[i], b[j])
        x[i][j] = amount
        a[i] -= amount
        b[j] -= amount

        if verbose:
            print(f"Добавлена базисная позиция ({i + 1}, {j + 1}),
x[{i + 1}][{j + 1}] = {round(amount, 6)}")

        if abs(a[i]) < TOL:
            a[i] = 0.0
```

```
        if abs(b[j]) < TOL:
            b[j] = 0.0

        if a[i] == 0 and i < m - 1:
            i += 1
        elif a[i] > 0 and b[j] == 0 and j < n - 1:
            j += 1
        else:
            break

    return x, basis


def compute_potentials(costs, basis):
    m = len(costs)
    n = len(costs[0])
    u = [None] * m
    v = [None] * n
    u[0] = 0.0

    changed = True
    while changed:
        changed = False
        for i, j in basis:
            if u[i] is not None and v[j] is None:
                v[j] = costs[i][j] - u[i]
                changed = True
            elif u[i] is None and v[j] is not None:
                u[i] = costs[i][j] - v[j]
                changed = True

    return u, v


def find_entering_position(costs, u, v, basis):
    m = len(costs)
    n = len(costs[0])
    basis_set = set(basis)
    candidates = []

    for i in range(m):
        for j in range(n):
            if (i, j) in basis_set:
                continue
            delta = costs[i][j] - u[i] - v[j]
            if delta < -TOL:
                candidates.append((i, j, delta))

    if not candidates:
        return None

    candidates.sort(key=lambda item: (item[0], item[1]))
    return candidates[0]


def build_cycle(basis, entering, m, n):
    """
    Строит единственный цикл после добавления entering.
```

```
    """
    graph = {}
    for i in range(m):
        graph[("r", i)] = []
    for j in range(n):
        graph[("c", j)] = []

    for i, j in basis:
        row_node = ("r", i)
        col_node = ("c", j)
        graph[row_node].append(col_node)
        graph[col_node].append(row_node)

    start = ("r", entering[0])
    finish = ("c", entering[1])

    queue = deque([start])
    parent = {start: None}

    while queue:
        node = queue.popleft()
        if node == finish:
            break
        for nxt in graph[node]:
            if nxt not in parent:
                parent[nxt] = node
                queue.append(nxt)

    node_path = []
    current = finish
    while current is not None:
        node_path.append(current)
        current = parent[current]
    node_path.reverse()

    cycle = [entering]
    for index in range(len(node_path) - 1):
        left = node_path[index]
        right = node_path[index + 1]
        if left[0] == "r":
            cycle.append((left[1], right[1]))
        else:
            cycle.append((right[1], left[1]))

    return cycle


def update_plan_by_cycle(plan, basis, cycle, verbose=True):
    minus_positions = [cycle[idx] for idx in range(1, len(cycle), 2)]
    theta = min(plan[i][j] for i, j in minus_positions)
    leaving_candidates = [(i, j) for i, j in minus_positions if
abs(plan[i][j] - theta) < TOL]
    leaving = min(leaving_candidates)

    if verbose:
        print(f"Цикл = {[(i + 1, j + 1) for i, j in cycle]}")
        print(f"theta = {round(theta, 6)}")
```

```
        print(f"Из базиса выходит позиция ({leaving[0] + 1},
{leaving[1] + 1})")

    new_plan = copy_matrix(plan)
    for idx, (i, j) in enumerate(cycle):
        if idx % 2 == 0:
            new_plan[i][j] += theta
        else:
            new_plan[i][j] -= theta
        if abs(new_plan[i][j]) < TOL:
            new_plan[i][j] = 0.0

    new_basis = basis[:] + [cycle[0]]
    new_basis.remove(leaving)
    return new_plan, new_basis


def total_cost(plan, costs):
    total = 0.0
    for i in range(len(plan)):
        for j in range(len(plan[0])):
            total += plan[i][j] * costs[i][j]
    return total


def solve_transport_problem(costs, supply, demand, verbose=True):
    if any(value < -TOL for value in supply) or any(value < -TOL for
value in demand):
        raise ValueError("Поставки и заявки должны быть
неотрицательными.")

    if abs(sum(supply) - sum(demand)) > TOL:
        raise ValueError("Транспортная задача несовместна: не
выполнено условие баланса.")

    if verbose:
        print("--- Метод северо-западного угла ---")

    plan, basis = northwest_corner_method(supply, demand,
verbose=verbose)

    if verbose:
        print_matrix("Начальный план перевозок", plan)
        print(f"Начальный базис B = {[(i + 1, j + 1) for i, j in
basis]}")

    iteration = 1
    while True:
        if verbose:
            print(f"\n--- Итерация улучшения {iteration} ---")

        u, v = compute_potentials(costs, basis)
        entering_info = find_entering_position(costs, u, v, basis)

        if verbose:
            print_vector_u = [0.0 if value is None else value for
value in u]
```

```
            print_vector_v = [0.0 if value is None else value for
value in v]
            print(f"u = {round_vector(print_vector_u)}")
            print(f"v = {round_vector(print_vector_v)}")

        if entering_info is None:
            if verbose:
                print("Все небазисные оценки неотрицательны. План
оптимален.")
            return plan, basis, total_cost(plan, costs)

        enter_i, enter_j, delta = entering_info
        if verbose:
            print(
                f"В базис входит позиция ({enter_i + 1}, {enter_j +
1}), "
                f"delta = {round(delta, 6)}"
            )

        cycle = build_cycle(basis, (enter_i, enter_j), len(costs),
len(costs[0]))
        plan, basis = update_plan_by_cycle(plan, basis, cycle,
verbose=verbose)

        if verbose:
            print_matrix("Новый план перевозок", plan)
            print(f"Новый базис B = {[(i + 1, j + 1) for i, j in
basis]}")

        iteration += 1


if __name__ == "__main__":
    # Пример из lab6_conditions.pdf
    costs_example = [
        [8.0, 4.0, 1.0],
        [8.0, 4.0, 3.0],
        [9.0, 7.0, 5.0],
    ]
    supply_example = [100.0, 300.0, 300.0]
    demand_example = [300.0, 200.0, 200.0]

    plan_result, basis_result, cost_result = solve_transport_problem(
        costs_example,
        supply_example,
        demand_example,
        verbose=True,
    )

    print("\n--- Результат ---")
    print_matrix("Оптимальный план перевозок", plan_result)
    print(f"Оптимальный базис B = {[(i + 1, j + 1) for i, j in
basis_result]}")
    print(f"Минимальная стоимость = {round(cost_result, 6)}")
```